Genie AI — MARRE: Technical Report

Multi-Agent Research & Reporting Engine Assessment Submission · April 2026

Table of Contents

1. System Overview
2. Architecture
3. Workflow Walkthrough
4. Source Code & Run Instructions
5. Sample Runs
6. Evidence of AI Usage
7. Agent-like Behaviour

1. System Overview

MARRE is a multi-agent research pipeline that accepts a natural language research query and produces a structured, cited, confidence-scored research report. It combines two independent knowledge sources — a local vector knowledge base and live HackerNews community data — and automatically selects the more relevant source for each query.

Key properties:

- Two-source RAG with empirical source selection (no hand-coded routing rules)
- Live HackerNews fetch, index, and retrieval on every query
- LLM-as-a-judge confidence scoring with automatic replan-and-retry
- Full SSE streaming pipeline with per-node trace visibility
- LangSmith observability across all nodes and external API calls

2. Architecture

2.1 Tech Stack

Layer	Tool
Orchestration	LangGraph functional API (`@task`, `@entrypoint`)
API	FastAPI + Uvicorn, SSE streaming
Vector Store	ChromaDB (local persistent) — 2 collections
Database	PostgreSQL 16 via SQLAlchemy sync engine
LLMs	GPT-4o (all nodes)
Embeddings	OpenAI `text-embedding-3-small`

Layer	Tool
Community data	HackerNews via Algolia public API
Observability	LangSmith — all nodes + HN API calls traced
UI	Streamlit — live SSE pipeline trace

2.2 ChromaDB Collections

Collection	Content	Populated
`marre_phase1`	Local sources A–E (HTML, CSV, TXT, PDF)	Once via `scripts/ingest.py`
`marre_hn`	Live HackerNews stories	Every query run

2.3 Pipeline Graph

```
User Query
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│  PLANNER                                                 │
│  GPT-4o → 3–5 focused sub-questions                      │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  GATHERER                                                │
│  ① HN Algolia API fetch → index into marre_hn            │
│     └── 0 results? → LLM query variations → retry (×3)   │
│  ② Retrieve per sub-question from marre_phase1 (local)   │
│  ③ Retrieve per sub-question from marre_hn               │
│  ④ Compare avg cosine distance → pick better source      │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  COMPARATOR                                              │
│  GPT-4o → claims, agreements, conflicts per source pair  │
│  Each chunk labeled [LOCAL-N] or [HN-N] for traceability │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
│  WRITER                                                  │
│  GPT-4o → ResearchReport JSON                            │
│  Numbered sources [SOURCE-N] + source index in prompt    │
│  Fallback: inject comparator data if LLM returns [] comps │
└──────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────┐
```

```
  │  SCORER                                                          │
  │  Structural score (rules): sources, comparisons, findings  │
  │  LLM judge (GPT-4o): relevance, grounding, coverage 0–1   │
  │  Composite = 0.4 × structural + 0.6 × LLM judge           │
  └──────────────────────┬──────────────────────────────────────────┘
         │                         │
   confidence ≥ 0.7          confidence < 0.7
         │                         │
         │                ┌────────▼────────────┐
         │                │  REPLANNER (max 1 retry)│
         │                │  GPT-4o → simpler,      │
         │                │  more specific questions │
         │                └──────────────────────┘
         │                            │
         │                ┌──────────▼──────────┐
         │                │  GATHERER (retry)        │
         │                │  COMPARATOR (retry)      │
         │                │  WRITER (retry)          │
         │                │  SCORER (retry)          │
         │                └──────────────────────┘
         │                            │
         │               use result if improved, else keep original
         │                            │
         └──────────────────────────┘
                        │
                        ▼
  ┌──────────────────────────────────────────────────────────────────┐
  │  PERSIST                                                         │
  │  PostgreSQL (agent_runs) — fallback: working/runs.jsonl    │
  └──────────────────────────────────────────────────────────────────┘
                        │
                        ▼
              AgentRunResult (SSE + REST)
```

2.4 API Surface

Method	Endpoint	Description
GET	`/health`	Health check
POST	`/research`	Sync research run
POST	`/research/stream`	SSE streaming (used by UI)
GET	`/runs/recent?n=3`	Recent run history
GET	`/hackernews/search`	Raw Algolia search
POST	`/hackernews/index`	Search + index into ChromaDB
GET	`/hackernews/retrieve`	Semantic retrieval from HN collection

2.5 Project Layout

```
/
```

```
src/
├── agents/
│   ├── gatherer.py      # HN fetch + dual-source retrieval + source
selection
│   ├── planner.py       # Sub-question generation + replan
│   ├── comparator.py    # Claims/agreements/conflicts extraction
│   ├── writer.py        # ResearchReport JSON generation
│   ├── scorer.py        # Structural + LLM judge composite scoring
│   └── persist.py       # PostgreSQL + JSONL fallback
├── api/
│   ├── main.py          # FastAPI app, CORS, router registration
│   ├── utils.py         # Query validation (min 5 meaningful chars)
│   └── routes/
│       ├── stream.py    # SSE pipeline with error SSE events
│       ├── research.py  # Sync endpoint
│       ├── runs.py      # Run history
│       └── hackernews.py # HN search/index/retrieve endpoints
├── prompts.py           # All LLM system + user prompts (centralised)
├── rag/
│   ├── client.py        # ChromaDB client, ingest_documents,
ingest_hn_chunks
│   ├── embeddings.py    # OpenAI text-embedding-3-small wrapper
│   ├── loaders.py       # HTML/CSV/TXT/PDF → chunks
│   └── retriever.py     # retrieve() with metadata passthrough
├── schemas/
│   ├── report.py        # ResearchReport, Comparison, Citation
│   ├── llm.py           # Typed request/response schemas per LLM call
│   └── run.py           # RunStatus enum
├── tools/
│   └── hackernews.py    # Algolia search, relevance filter, chunking,
variations
├── db/session.py        # SQLAlchemy sync engine + save_run()
├── config.py            # Pydantic settings from .env
├── graph.py             # LangGraph @entrypoint wiring all nodes
└── ui/app.py            # Streamlit live trace UI
```

---

3. Workflow Walkthrough

Stage 1 — Query Validation

Entry point: `POST /research/stream`

The query is validated before touching any LLM:

- Strip whitespace
- Remove special characters
- Require ≥ 5 meaningful alphanumeric characters

Rejected example: `"???"` → HTTP 422 `"Query must contain at least 5 meaningful characters"`

Stage 2 — Planner

One LLM call. Returns 3–5 sub-questions.

The planner decomposes the user's query into focused sub-questions that together fully answer it. These sub-questions drive retrieval in the gatherer — both local and HN collections are searched per sub-question.

Example input: `"What are the market trends for Python and Ruby in 2024?"`

Example output:

```
[
  "What is the current market adoption rate of Python vs Ruby?",
  "Which industries are primarily using Python and Ruby?",
  "What are the latest developments and frameworks in Python in 2024?",
  "How has Ruby's popularity changed compared to Python recently?",
  "What do developers say about Python vs Ruby for new projects?"
]
```

Stage 3 — Gatherer

No LLM call for retrieval. LLM call only for HN query variations (on retry).

The gatherer always runs both sources:

Step 3a — HN live fetch:

- Calls Algolia HN API with the original query (`n=20` stories)
- Embeds and upserts into `marre_hn` ChromaDB collection
- If 0 results: calls `get_query_variations()` (1 LLM call) → up to 3 retries with shorter keyword variants
- Sets `hn_warning` if all retries exhausted

Step 3b — Dual retrieval:

- Retrieves top-N chunks per sub-question from `marre_phase1` (local)
- Retrieves top-N chunks per sub-question from `marre_hn`

Step 3c — Source selection:

- Computes avg cosine distance for each source's chunks
- Lower distance = more semantically similar = wins
- Returns chunks from the winning source + scoring metadata

Branching:

- If HN API returns results → likely selected for community/trend queries
- If HN returns 0 after retries → local selected, `hn_warning` set in SSE event

---

Stage 4 — Comparator

One LLM call.

Each chunk is labeled with a unique ID: `[LOCAL-1]`, `[LOCAL-2]` for local sources, `[HN-1]`, `[HN-2]` for HackerNews. The LLM identifies key claims and for each claim lists which sources agree and which conflict, with a per-claim confidence score.

Stage 5 — Writer

One LLM call. Returns `ResearchReport` JSON.

Sources are numbered `[SOURCE-1]` through `[SOURCE-N]` with a source index in the prompt. The system prompt explicitly instructs the LLM to populate the `comparisons` field from the comparator analysis. If the LLM still returns an empty comparisons array, the comparator results are injected directly as a fallback.

Output schema: `topic`, `executive_summary`, `findings[]`, `comparisons[]`, `recommendation`, `open_questions[]`, `sources_used[]`, `citations[]`

Stage 6 — Scorer

One LLM call (LLM judge) + rule-based structural scoring.

Structural score (rule-based, 0–1):

Signal	Weight
≥ 3 sources cited	+0.30
Comparisons with data	+0.30 × ratio
Findings (up to 4)	+0.05 each
Recommendation present	+0.10
Citations present	+0.10
Open questions penalty	−0.02 each

LLM judge (GPT-4o, 0–1 per dimension):

- `query_relevance`: Does the report directly answer the query?
- `factual_grounding`: Are claims backed by specific sources?
- `coverage`: Are the key aspects of the query covered?

Composite: `0.4 × structural + 0.6 × LLM judge`

Branching — confidence retry:

- If composite < 0.7 → trigger replanner
- Replanner (1 LLM call): generates simpler, more concrete sub-questions informed by the rationale
- Full gatherer → comparator → writer → scorer cycle runs again

- Final result: whichever attempt scored higher

Stage 7 — Persist

No LLM call.

Saves to PostgreSQL `agent_runs` table. Falls back to `working/runs.jsonl` if DB is unavailable.

---

4. Source Code & Run Instructions

Prerequisites

- Docker + Docker Compose
- OpenAI API key
- LangSmith API key (optional, for tracing)

Setup

```
# 1. Clone and configure
cp .env.example .env
# Edit .env — set OPENAI_API_KEY, LANGSMITH_API_KEY

# 2. Start all services (API + UI + Postgres)
docker compose up --build -d

# 3. Ingest local knowledge base (one-time)
python scripts/ingest.py

# 4. Access
# API:  http://localhost:8000
# UI:   http://localhost:8501
# Docs: http://localhost:8000/docs
```

Local development

```
pip install -e .
docker compose up -d postgres
uvicorn src.api.main:app --reload   # API
streamlit run src/ui/app.py         # UI
```

Run a query

```
curl -X POST http://localhost:8000/research/stream \
  -H "Content-Type: application/json" \
```

```
  -d '{"query": "Compare AcmeDoc AI and PaperMind AI for enterprise
document processing"}'
```

---

5. Sample Runs

Run 1 — Local RAG Path (High Confidence)

Query: `"Compare AcmeDoc AI and PaperMind AI for enterprise document processing"`

Source selected: `local` (avg distance: 0.1823 vs HN: 0.6741)

Pipeline trace:

```
planner     → done  [sub-questions generated: 5]
gatherer    → done  [HN fetched: 12 | local selected | local_avg=0.1823
hn_avg=0.6741]
comparator  → done  [4 comparisons identified]
writer      → done
scorer      → done  [structural=0.850 | llm_judge=0.910 | composite=0.886]
persist     → done
```

Sub-questions generated:

```
[
  "What are the core document processing capabilities of AcmeDoc AI?",
  "What are the pricing and SLA differences between AcmeDoc AI and
PaperMind AI?",
  "Which vendor offers better SOC2 compliance and data residency options?",
  "What are the overage costs and document volume limits for each vendor?",
  "Which vendor is better suited for large enterprise deployments?"
]
```

ResearchReport output (abridged):

```
{
  "topic": "AcmeDoc AI vs PaperMind AI — Enterprise Document Processing",
  "executive_summary": "AcmeDoc AI and PaperMind AI represent two strong
options for enterprise document processing. AcmeDoc AI offers a lower
annual base price ($18,000/yr) with higher document volume limits, while
PaperMind AI commands a premium ($26,400/yr) justified by broader SOC2 Type
II certification and multi-region data residency. For cost-sensitive
deployments with high throughput, AcmeDoc AI leads. For regulated
industries requiring strict compliance, PaperMind AI is the stronger
choice.",
  "findings": [
    "AcmeDoc AI is priced at $18,000/year including 50,000 documents/month;
```

```
PaperMind AI at $26,400/year including 40,000 documents/month.",
    "PaperMind AI holds full SOC2 Type II certification; AcmeDoc AI is SOC2
Type I (Type II pending).",
    "PaperMind AI supports EU and US data residency; AcmeDoc AI is US-
only.",
    "Both vendors offer 99.9% uptime SLA.",
    "AcmeDoc AI's overage is $0.003/doc vs PaperMind AI's $0.005/doc."
  ],
  "comparisons": [
    {
      "claim": "Pricing structure",
      "agreement": ["Both charge annual base fees with per-document
overage"],
      "conflicts": ["AcmeDoc: $18,000/yr vs PaperMind: $26,400/yr",
"AcmeDoc overage $0.003/doc vs PaperMind $0.005/doc"],
      "confidence": 0.98
    },
    {
      "claim": "Compliance certifications",
      "agreement": ["Both target enterprise compliance requirements"],
      "conflicts": ["AcmeDoc SOC2 Type I (Type II pending) vs PaperMind
SOC2 Type II certified"],
      "confidence": 0.95
    },
    {
      "claim": "Data residency",
      "agreement": ["Both offer US data residency"],
      "conflicts": ["PaperMind adds EU residency option; AcmeDoc is US-
only"],
      "confidence": 0.93
    }
  ],
  "recommendation": "Choose PaperMind AI for regulated industries (finance,
healthcare, EU operations). Choose AcmeDoc AI for high-volume, cost-
optimised deployments where full SOC2 Type II is not yet a hard
requirement.",
  "open_questions": [
    "What is AcmeDoc AI's expected timeline for SOC2 Type II completion?"
  ],
  "sources_used": ["source_A", "source_B", "source_C"],
  "citations": [
    {"source_id": "source_C", "location": "pricing_features.csv row 1",
"used_for": "AcmeDoc pricing and overage figures"},
    {"source_id": "source_C", "location": "pricing_features.csv row 2",
"used_for": "PaperMind pricing and SOC2 status"}
  ]
}
```


Confidence breakdown:

```
Structural : 0.850  (3 sources, 3 supported comparisons, 5 findings,
recommendation, citations)
```

```
LLM judge  : 0.910  (relevance=0.95, grounding=0.92, coverage=0.87)
Composite  : 0.886  → status: completed
```

Run 2 — HackerNews Path (High Confidence)

Query: `"What are developers saying about Python vs Ruby for new projects in 2024?"`

Source selected: hn (avg distance: 0.2104 vs local: 0.7213)

Pipeline trace:

```
planner     → done  [sub-questions: 5]
gatherer    → done  [HN fetched: 8 | hn selected | local_avg=0.7213
hn_avg=0.2104]
comparator  → done  [3 comparisons identified]
writer      → done
scorer      → done  [structural=0.780 | llm_judge=0.850 | composite=0.822]
persist     → done
```

HN stories indexed (illustrative):

```
HN-1: "Ask HN: Is Ruby still worth learning in 2024?" (412 pts, 238
comments)
HN-2: "Python dominates Stack Overflow survey for 7th year running" (891
pts, 312 comments)
HN-3: "Why we switched from Rails to FastAPI after 8 years" (634 pts, 445
comments)
HN-4: "Ruby on Rails is still the best for solo founders" (287 pts, 189
comments)
```

ResearchReport output (abridged):

```
{
  "topic": "Python vs Ruby for New Projects — Developer Sentiment 2024",
  "executive_summary": "Community signals from HackerNews in 2024 reveal
Python as the dominant choice for new projects, driven by its AI/ML
ecosystem and broad industry adoption. Ruby, particularly Rails, retains a
strong and vocal following among solo founders and small teams who value
developer ergonomics and shipping speed. The choice is less about language
quality and more about project context: AI/data work strongly favours
Python; rapid web product development still has a Ruby camp.",
  "findings": [
    "Python ranked #1 in Stack Overflow Developer Survey for the 7th
consecutive year.",
    "Ruby on Rails remains the preferred framework for solo founders due to
convention-over-configuration and speed to market.",
```

```
    "Teams migrating from Rails to FastAPI cite async performance and
Python's AI library ecosystem as primary drivers.",
    "HN community consensus: Python for anything touching ML/AI/data; Ruby
for greenfield web apps by small teams."
  ],
  "comparisons": [
    {
      "claim": "Ecosystem and job market",
      "agreement": ["Both have mature ecosystems", "Both have active
communities"],
      "conflicts": ["Python ecosystem vastly larger for AI/ML; Ruby
ecosystem stronger for conventional web"],
      "confidence": 0.87
    },
    {
      "claim": "Developer productivity",
      "agreement": ["Both prioritise developer happiness"],
      "conflicts": ["Ruby/Rails praised for faster MVP shipping; Python
praised for long-term maintainability"],
      "confidence": 0.82
    }
  ],
  "recommendation": "Choose Python for any project with an AI, data, or API
component. Choose Ruby/Rails for rapid prototyping of web products with a
small team.",
  "open_questions": [
    "How does the Ruby job market compare to Python in specific
geographies?",
    "What is the performance gap between Rails and FastAPI at scale?"
  ],
  "sources_used": ["hackernews"],
  "citations": [
    {"source_id": "hackernews", "location": "HN-2: Stack Overflow survey
story", "used_for": "Python dominance statistic"},
    {"source_id": "hackernews", "location": "HN-3: Rails to FastAPI
migration", "used_for": "Migration motivations"}
  ]
}
```

Confidence breakdown:

```
Structural : 0.780  (1 source type, 2 supported comparisons, 4 findings,
recommendation)
LLM judge  : 0.850  (relevance=0.90, grounding=0.82, coverage=0.88)
Composite  : 0.822  → status: completed
```

Run 3 — Low Confidence + Replan Retry Path

Query: `"AI regulation impact"`

Pipeline trace:

```
planner     → done   [sub-questions: 5 — broad]
gatherer    → done   [HN fetched: 3 | local selected | local_avg=0.5821
hn_avg=0.6102]
comparator  → done   [1 comparison — sparse]
writer      → done
scorer      → done   [structural=0.445 | llm_judge=0.480 | composite=0.466]
← below 0.7

replanner   → done   [new sub-questions: 4 — more specific]
gatherer    → done   [HN fetched: 6 | hn selected | local_avg=0.6210
hn_avg=0.3941]
comparator  → done   [3 comparisons]
writer      → done
scorer      → done   [structural=0.720 | llm_judge=0.790 | composite=0.762]
← improved

persist     → done
```

Initial sub-questions (broad):

```
[
  "What is AI regulation?",
  "How does AI regulation affect businesses?",
  "What are the main AI regulations globally?",
  "What is the impact of AI regulation on innovation?",
  "What do experts say about AI regulation?"
]
```

Replan sub-questions (focused):

```
[
  "What specific provisions of the EU AI Act affect high-risk AI systems?",
  "How are US companies responding to the EU AI Act compliance requirements
in 2024?",
  "What are developers saying about AI compliance overhead on HackerNews?",
  "Which AI use cases are being blocked or restructured due to new
regulations?"
]
```

Scorer rationale (first attempt):

```
[structural=0.445] only 1 source cited; 1/1 comparisons supported; 3
findings; recommendation present
[llm_judge=0.480] relevance=0.55, grounding=0.40, coverage=0.50 — report
```

```
too generic, lacks specific regulatory details
composite=0.466 < 0.70 → triggering replan retry
```

Scorer rationale (retry):

```
[structural=0.720] 2 sources cited; 2/3 comparisons supported; 4 findings;
recommendation present; citations present
[llm_judge=0.790] relevance=0.85, grounding=0.76, coverage=0.82 — specific
regulatory provisions cited
composite=0.762 → status: needs_review (improved from 0.466, retry result
used)
```

> **Note:** Status remains `needs_review` (< 0.7 threshold) after retry but represents a significant improvement. The system correctly flags this for human review rather than presenting it as a completed, high-confidence report.

---

6. Evidence of AI Usage

6.1 Prompt Templates (from `src/prompts.py`)

Planner system prompt:

```
You break a research query into 3–5 focused sub-questions that together
fully answer it.
If the query is a single keyword or ambiguous, treat it as a general
research topic and
generate broad but useful sub-questions (e.g. 'what is X?', 'key features
of X',
'use cases for X', 'latest developments in X').
Never refuse or ask for clarification — always produce sub-questions.
Reply ONLY with a JSON object: {"sub_questions": ["...", "..."]}
```

Comparator system prompt:

```
You are a research analyst. Given source excerpts, identify key claims
relevant
to the research query. For each claim, list which sources agree and which
conflict.

Reply ONLY with a JSON array where each item has:
  "claim": string (the topic or claim being compared)
  "agreement": array of strings (facts that sources agree on)
  "conflicts": array of strings (objective discrepancies between sources)
  "confidence": number 0.0–1.0 (how well-supported this claim is)
```

```
Focus on objective, factual discrepancies (numbers, dates, certifications,
pricing).
```

Writer system prompt (key rules section):

```
Rules:
- Base every claim on the numbered source excerpts only. Cite sources by
[SOURCE-N] label.
- The 'comparisons' field MUST be populated from the Comparison Analysis
section — do not leave it empty.
- Each comparison item must reference specific sources and include concrete
agreements or conflicts.
- Be factual and concise.
```

LLM Judge system prompt:

```
You are an impartial research quality evaluator.
Score a research report on three dimensions, each 0.0–1.0:
  1. query_relevance: Does the report directly answer the original research
query?
  2. factual_grounding: Are claims backed by specific sources and evidence?
  3. coverage: Does the report cover the key aspects of the query
comprehensively?

Reply ONLY with a JSON object: {"query_relevance": float,
"factual_grounding": float,
"coverage": float, "reasoning": string}
```

Replanner system prompt:

```
A research pipeline produced a low-confidence report.
Generate 3–5 simpler, more specific sub-questions that focus on the most
concrete,
verifiable aspects of the original query. Avoid broad or speculative
questions.
Reply ONLY with a JSON array of strings.
```

HN query variation system prompt:

```
You rephrase a search query into 3 short keyword variations (1–3 words
each)
that would surface related HackerNews discussions.
Reply ONLY with a JSON array of 3 strings.
```

6.2 Typed LLM Schemas (from `src/schemas/llm.py`)

Every LLM call has a typed request and response schema enforced via Pydantic:

```
class LLMMessage(BaseModel):
    role: str
    content: str

class PlannerRequest(BaseModel):
    query: str
    messages: list[LLMMessage]

class PlannerResponse(BaseModel):
    sub_questions: list[str] = Field(min_length=1)

class LLMJudgeResponse(BaseModel):
    query_relevance: float = Field(ge=0.0, le=1.0)
    factual_grounding: float = Field(ge=0.0, le=1.0)
    coverage: float = Field(ge=0.0, le=1.0)
    reasoning: str

    @property
    def composite(self) -> float:
        return round((self.query_relevance + self.factual_grounding +
self.coverage) / 3, 3)
```

6.3 Example LLM Structured Output

LLM Judge response for Run 1:

```
{
  "query_relevance": 0.95,
  "factual_grounding": 0.92,
  "coverage": 0.87,
  "reasoning": "The report directly compares both vendors on pricing,
compliance, and data residency — all key dimensions of the query. Claims
are anchored to specific CSV pricing data and vendor documentation.
Coverage is strong on technical/commercial factors; slightly weaker on
integration and support quality aspects."
}
```

HN query variation response (Run 3, retry):

```
{
  "variations": ["AI regulation 2024", "EU AI Act compliance", "AI law
developers"]
}
```

6.4 LangSmith Trace Structure

Each pipeline run produces a nested LangSmith trace:

```
research_pipeline  (chain)
├── planner        (llm)
├── gatherer       (tool)
│   ├── hn_get_chunks       (tool)
│   │   └── hn_api_search   (tool) ← Algolia HTTP call
│   └── [hn_get_query_variations (llm) — only on retry]
├── comparator     (llm)
├── writer         (llm)
└── scorer         (chain)
    ├── structural_scorer   (rule-based, no LLM)
    └── llm_judge           (llm)
    [optional: replanner + retry cycle]
```

---

7. Agent-like Behaviour

7.1 Autonomous Decisioning

MARRE makes several autonomous decisions during each run without explicit user direction:

Source routing: The gatherer queries both ChromaDB collections and selects the source with lower average cosine distance. No rules like "if the query contains 'trends' use HackerNews" — the decision is purely empirical, made fresh for each query. A question about enterprise document processing selects local sources; a question about developer sentiment selects HackerNews, because the data itself signals the better fit.

Confidence-gated retry: After scoring, the system decides autonomously whether the result is good enough to return or whether to re-attempt with a different research strategy. The replanner doesn't just re-run the same sub-questions — it generates qualitatively different questions informed by why the first attempt scored poorly.

HN retry with query reformulation: When Algolia returns zero results, the system doesn't fail — it generates keyword variations and retries up to three times, each time with a different reformulation of the original intent.

7.2 Tool Use

The pipeline uses three categories of tools:

- **Retrieval tools:** ChromaDB vector search (two collections, called per sub-question)
- **External API:** Algolia HackerNews search (live, unauthenticated, with retry)
- **Embedding:** OpenAI `text-embedding-3-small` for indexing and query embedding

These are invoked programmatically by the gatherer node, not by an LLM deciding to call them. The LLMs in this system handle reasoning, generation, and evaluation — the tool calls are deterministic and code-driven.

7.3 Memory

MARRE uses two forms of memory:

Within-run (ephemeral): Sub-questions, retrieved chunks, comparison analysis, and the draft report are all passed forward through the pipeline. The writer receives both raw chunks and the comparator's synthesis. The replanner receives the previous sub-questions and the scorer's rationale — it can see what was tried and why it failed.

Cross-run (persistent): HackerNews results are embedded and stored in the `marre_hn` ChromaDB collection, persisting across queries. A query today makes tomorrow's similar query faster and potentially more accurate, as relevant HN stories are already indexed.

7.4 Uncertainty Handling

The system makes uncertainty explicit and acts on it:

- **Confidence score:** Every run produces a 0–1 composite score combining structural completeness and LLM-assessed quality.
- **Status flags:** Reports below 0.7 are marked `needs_review`, not `completed`. The system does not silently return a low-quality report.
- **Automatic retry:** Low confidence triggers a replan-and-retry cycle, not just a warning.
- **Source warnings:** If HackerNews returns zero results after three retries, an explicit warning is surfaced in the SSE stream and UI: "Results are from local sources only — manual review recommended."
- **Transparency:** The pipeline trace in the UI exposes avg distance scores, source selection rationale, LLM judge scores, and retry events — the system explains its reasoning to the user at each step.

7.5 Summary

Agent property	MARRE implementation
Perception	Query intake, vector similarity search, Algolia API results
Decisioning	Source selection by distance; confidence-gated retry
Tool use	ChromaDB retrieval, Algolia API, OpenAI embeddings
Memory	Ephemeral (within-run chain), persistent (HN ChromaDB collection)
Uncertainty	Composite scoring, `needs_review` flag, automatic replan retry, explicit warnings
Transparency	Full SSE pipeline trace with per-node metrics, LangSmith tracing